

Introducción a VHDL

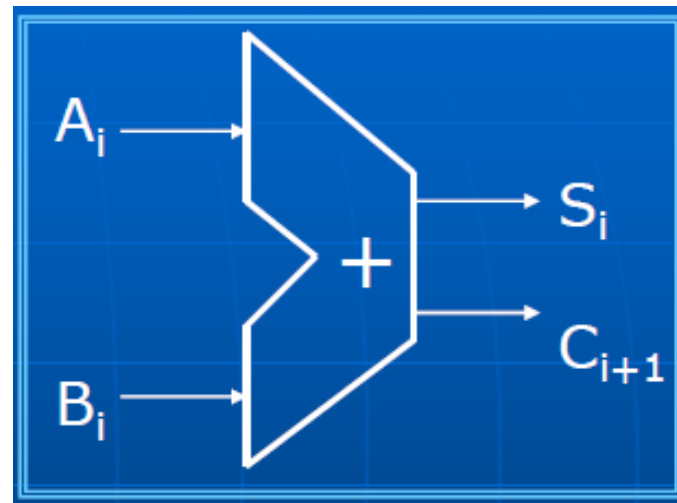
Matías Javier Oliva
Pablo García

Circuitos aritméticos

- Las operaciones aritméticas de las que felizmente nos abstraemos cuando programamos software tienen que realizarse físicamente en algún lado.
- La unidad aritmético-lógica de los procesadores implementa estas operaciones con circuitos aritméticos.
- Existen distintas arquitecturas para cada una de las operaciones básicas, que pueden implementarse con lógica combinacional o secuencial.
- En este curso veremos circuitos para implementar sumas restas y multiplicaciones
- Los circuitos para implementar divisiones resultan bastante complejos y en general se intentan evitar en hardware, ya que pueden implementarse en software usando los elementos básicos (pensar en un set de instrucciones de un microprocesador).
- Tener en cuenta también que si se utilizan librerías aritméticas con el operador “+”, por ejemplo se pierde control sobre la lógica subyacente.

Circuitos sumadores: Half-Adder

A_i	B_i	C_{i+1}	S_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

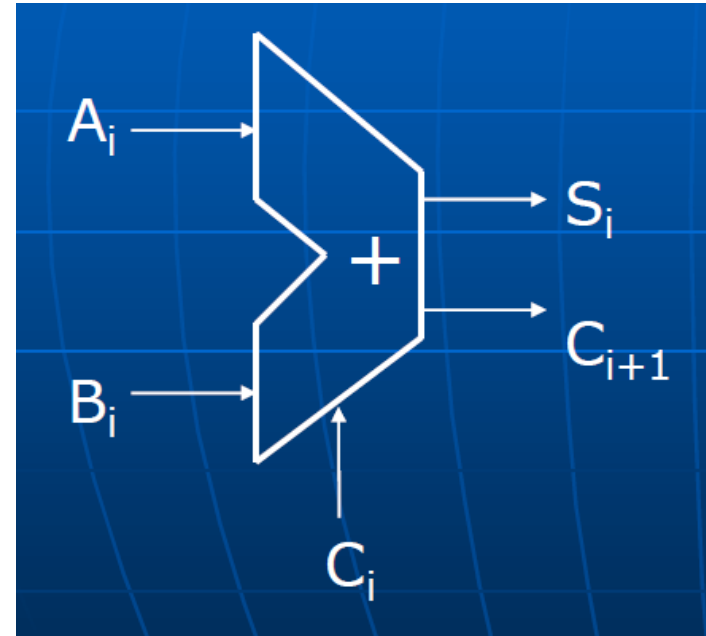


$$S_i = A_i \oplus B_i$$

$$C_{i+1} = A_i B_i$$

Circuitos sumadores: Full-Adder

C_i	A_i	B_i	C_{i+1}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$

Circuitos sumadores: Full-Adder

```
library ieee;
use ieee.std_logic_1164.all;

entity FA is

port
(
  A : in std_logic;
  B : in std_logic;

  Ci : in std_logic;

  S : out std_logic;
  Co : out std_logic
);

end entity;
```

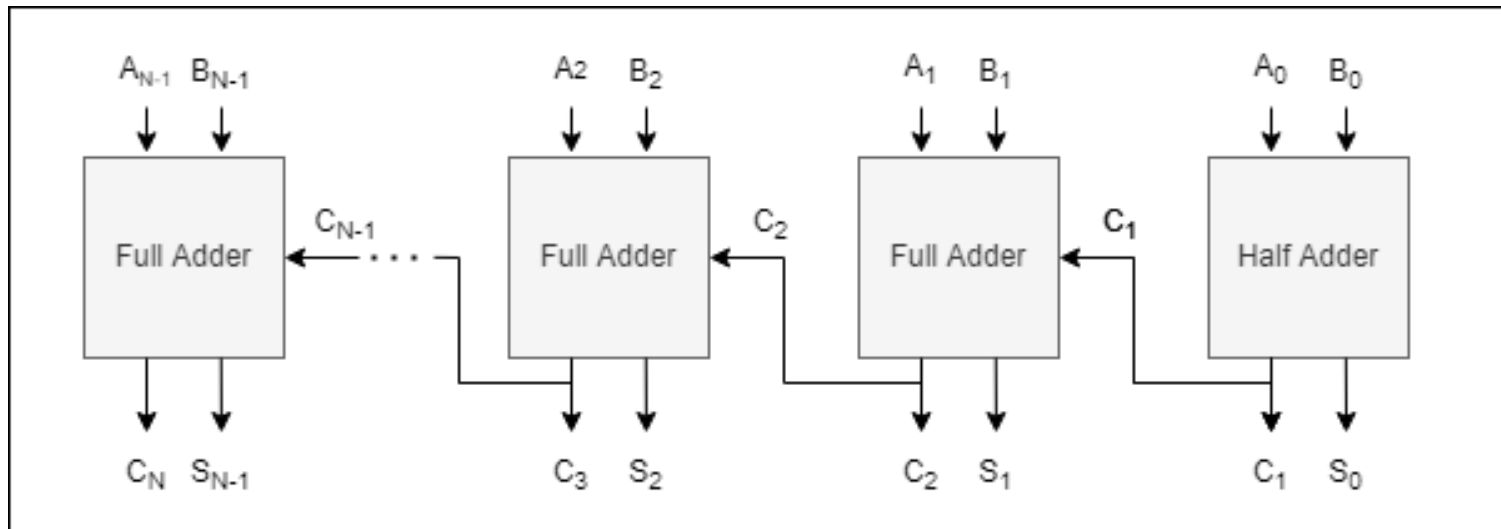
```
architecture sum of FA is
begin

  S <= A xor B xor Ci;
  Co <= (A and B) or (A and Ci) or (B and Ci);

end sum;
```

Circuitos sumadores: Ripple Carry

- Conectando muchos FA entre si se puede armar el más sencillo de los sumadores de N bits: El Ripple Carry de N bits

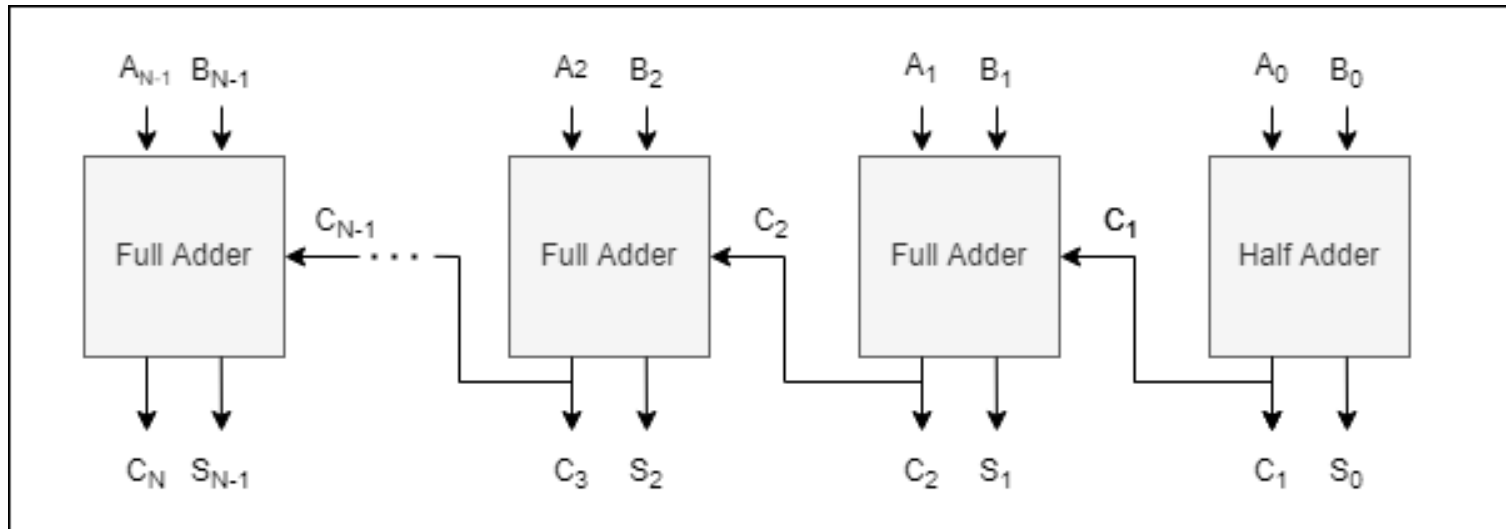


$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$

Tiempo de un XOR: 2τ
 S0 tarda 4τ . Los otros tardan 2τ .
 Ci tarda 2τ

Circuitos sumadores: Ripple Carry



$$t_{pd}(C_{i+1}) = t_{pd}(C_i) + 2\tau$$

$$t_{pd}(S_i) = t_{pd}(C_i) + 2\tau$$

Ejemplo N=8

$$t_{pd}(C_{i+1}) = t_{pd}(S_i) = 16\tau$$

- El último FF Impone la máxima velocidad a la que puede operarse el clock.
- Mientras más bits más lenta se hace la operación.

Circuitos sumadores: Ripple Carry

```

library ieee;
use ieee.std_logic_1164.all;

entity Clase5_a is
generic( N : integer := 8 );
port
(
  clk : in std_logic;
  reset : in std_logic;
  A : in std_logic_vector (N-1 downto 0);
  B : in std_logic_vector (N-1 downto 0);

  S : out std_logic_vector (N-1 downto 0)
);
end entity;

```

```

architecture arch of Clase5_a is
component FA is
  port
  (
    A : in std_logic;
    B : in std_logic;
    Ci : in std_logic;
    S : out std_logic;
    Co : out std_logic
  );
end component;

component registro is
  generic( N : integer := 4);
  port (
    clk : in std_logic;
    D : in std_logic_vector (N-1 downto 0);
    reset : in std_logic;
    set : in std_logic;
    enable : in std_logic;
    Q : out std_logic_vector (N-1 downto 0);
    Q_n : out std_logic_vector (N-1 downto 0)
  );
end component;

```

El componente registro son
N ff tipo D.

Se agrega porque Para ver
el timing es necesario ver
transiciones entre FF

Circuitos sumadores: Ripple Carry

```

signal C : std_logic_vector (N downto 0);
signal reg_a,reg_b,reg_s,signal_s : std_logic_vector (N-1 downto 0);
begin

ff_a: registro generic map (N) port map (clk,A,reset,'0','1',reg_a,open);
ff_b: registro generic map (N) port map (clk,B,reset,'0','1',reg_b,open);

C(0) <= '0';

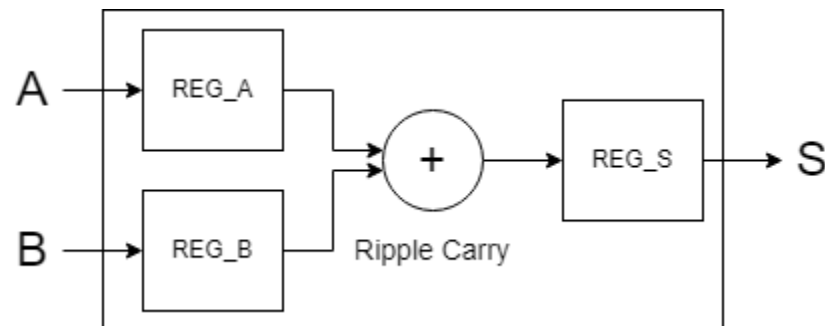
gen:
for i in 0 to N-1 generate
    fa_i: FA port map ( reg_a(i),reg_b(i),C(i),signal_s(i),C(i+1));
end generate;

ff_s: registro generic map (N) port map (clk,signal_s,reset,'0','1',reg_s,open);

S <= reg_s;

end arch;

```



Circuitos sumadores: Ripple Carry

N= 4

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	340.25 MHz	250.0 MHz	clock	limit due to minimum period restriction (max I/O toggle rate)

N= 8

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	212.81 MHz	212.81 MHz	clock	

N= 16

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	122.94 MHz	122.94 MHz	clock	

N= 32

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	73.42 MHz	73.42 MHz	clock	

N= 64

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax		
1	57.11 MHz	57.11 MHz		

Total logic elements	213 / 29,440 (< 1 %)
Total combinational functions	162 / 29,440 (< 1 %)
Dedicated logic registers	192 / 29,440 (< 1 %)
Total registers	192

Circuitos sumadores: Look-ahead Carry

- La principal limitación del RippleCarry es que para que la suma se compute se debe esperar a que se propague el Carry desde el primer bit.
- Para optimizar esto se puede implementar una predicción de Carry en paralelo, aprovechando que en realidad el Carry solo depende de los bits de A y B que son conocidos desde el principio.
- Un problema de esto es que se necesita mucha lógica para implementar la predicción de Carry en paralelo para N grandes. En la práctica se suele utilizar hasta $N=4$ y después combinar con otras técnicas.
- Para predecir el Carry es útil generar señales extra Generate y Propagate.

Circuitos sumadores: Look-ahead Carry

Generate: $G_i = A_i B_i$ Se pone en 1 si la etapa **va a generar** Carry

Propagate: $P_i = A_i \oplus B_i$ Se pone en 1 si la etapa **va a propagar** un Carry que eventualmente reciba

- Con estas señales se puede calcular la suma directamente

$C_{i+1} = G_i + P_i C_i$ El Carry va a ser 1 si esa etapa **lo genera** o si lo **propaga y lo tengo en la entrada**

$S_i = P_i \oplus C_i$ Se pone en 1 si **hay Carry y no se propaga** o si **se propaga y no hay Carry**

Circuitos sumadores: Look-ahead Carry

Generate: $G_i = A_i B_i$ Se pone en 1 si la etapa **va a generar** Carry

Propagate: $P_i = A_i \oplus B_i$ Se pone en 1 si la etapa **va a propagar**
un Carry que eventualmente reciba

- Con estas señales se puede calcular la suma directamente

$C_{i+1} = G_i + P_i C_i$ El Carry va a ser 1 si esa etapa **lo genera** o si lo **propaga y lo tengo en la entrada**

$S_i = P_i \oplus C_i$ Se pone en 1 si **hay Carry y no se propaga** o si **se propaga y no hay Carry**

Ejemplo: Look-ahead Carry de 4 bits

$$C1 = G0 + P0 C0$$

$$C2 = G1 + P1 C1 = G1 + P1 G0 + P1 P0 C0$$

$$C3 = G2 + P2 C2 = G2 + P2 G1 + P2 P1 G0 + P2 P1 P0 C0$$

$$C4 = G3 + P3 C3 = G3 + P3 G2 + P3 P2 G1 + P3 P2 P1 G0 + P3 P2 P1 P0 C0$$

$$S_i = P_i \oplus C_i$$

- Todo se puede calcular en paralelo! Ninguna etapa se queda esperando.
- Aumentar la cantidad de bits incrementa exponencialmente la cantidad de hardware necesario
- Se puede usar el de 4 bits y combinar con otras etapas tipo Ripple Carry para incrementar la cantidad de bits

Implementación: Look-ahead Carry de 4 bits

```
architecture Behavioral of LookAheadCarry4bits is
    signal G, P : std_logic_vector(3 downto 0); -- Generate and Propagate
    signal C : std_logic_vector(4 downto 0);    -- Carry vector (C(0) = 0)
begin

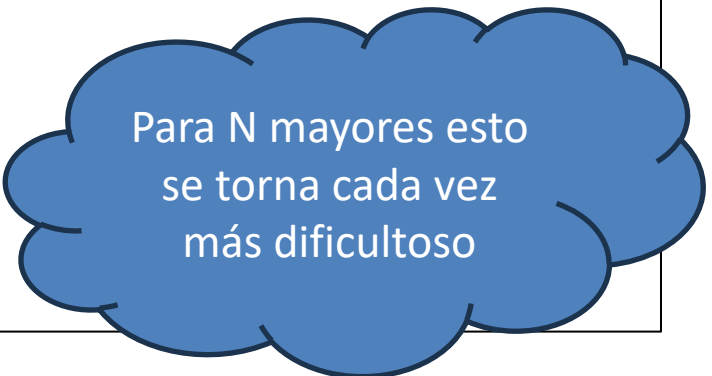
    -- Inicialización del carry-in
    C(0) <= cin;

    -- Generate y Propagate
    G <= A and B;
    P <= A xor B;

    -- Carry Look-Ahead Logic (expansión completa)
    C(1) <= G(0) or (P(0) and C(0));
    C(2) <= G(1) or (P(1) and G(0)) or (P(1) and P(0) and C(0));
    C(3) <= G(2) or (P(2) and G(1)) or (P(2) and P(1) and G(0)) or (P(2) and P(1) and P(0) and C(0));
    C(4) <= G(3) or (P(3) and G(2)) or (P(3) and P(2) and G(1)) or
        (P(3) and P(2) and P(1) and G(0)) or
        (P(3) and P(2) and P(1) and P(0) and C(0));

    -- Suma final
    S(0) <= P(0) xor C(0);
    S(1) <= P(1) xor C(1);
    S(2) <= P(2) xor C(2);
    S(3) <= P(3) xor C(3);

    cout <= C(4);
end architecture;
```



Para N mayores esto
se torna cada vez
más dificultoso

Implementación:

Combinación de LAC en formato RippleCarry

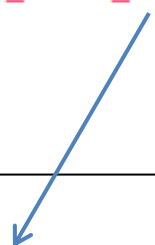
```
architecture Structural of LookAheadCarry is

    constant NUM_BLOCKS : integer := N / 4;

    component LookAheadCarry4bits is
    port (
        A : in std_logic_vector(3 downto 0);
        B : in std_logic_vector(3 downto 0);
        cin : in std_logic;
        S : out std_logic_vector(3 downto 0);
        cout : out std_logic
    );
end component;

-- Tipo para array de std_logic_vector(3 downto 0)
type slv4_array is array (natural range <>) of std_logic_vector(3 downto 0);

-- Arreglo para carries entre bloques
signal C : std_logic_vector(NUM_BLOCKS downto 0);
signal S_blocks : slv4_array(0 to NUM_BLOCKS-1);
```



Se define un tipo de señal array para agarrar los bloques de 4 bits de cada instancia de Look Ahead Carry

Implementación:

Combinación de LAC en formato RippleCarry

```
begin

    -- Carry inicial
    C(0) <= '0';

    -- Instancias en ripple
    gen_blocks: for i in 0 to NUM_BLOCKS-1 generate
        U_CLA4: LookAheadCarry4bits
            port map (
                A      => A(4*i+3 downto 4*i),
                B      => B(4*i+3 downto 4*i),
                Cin     => C(i),
                S       => S(4*i+3 downto 4*i),
                cout    => C(i+1)
            );
    end generate;

end architecture;
```

Circuitos sumadores: Look-ahead Carry

N= 4

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	385.51 MHz	250.0 MHz	clock	limit due to minimum period restriction (max I/O toggle rate)

N= 8

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	281.53 MHz	250.0 MHz	clock	limit due to minimum period restriction (max I/O toggle rate)

N= 16

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	150.85 MHz	150.85 MHz	clock	

N= 32

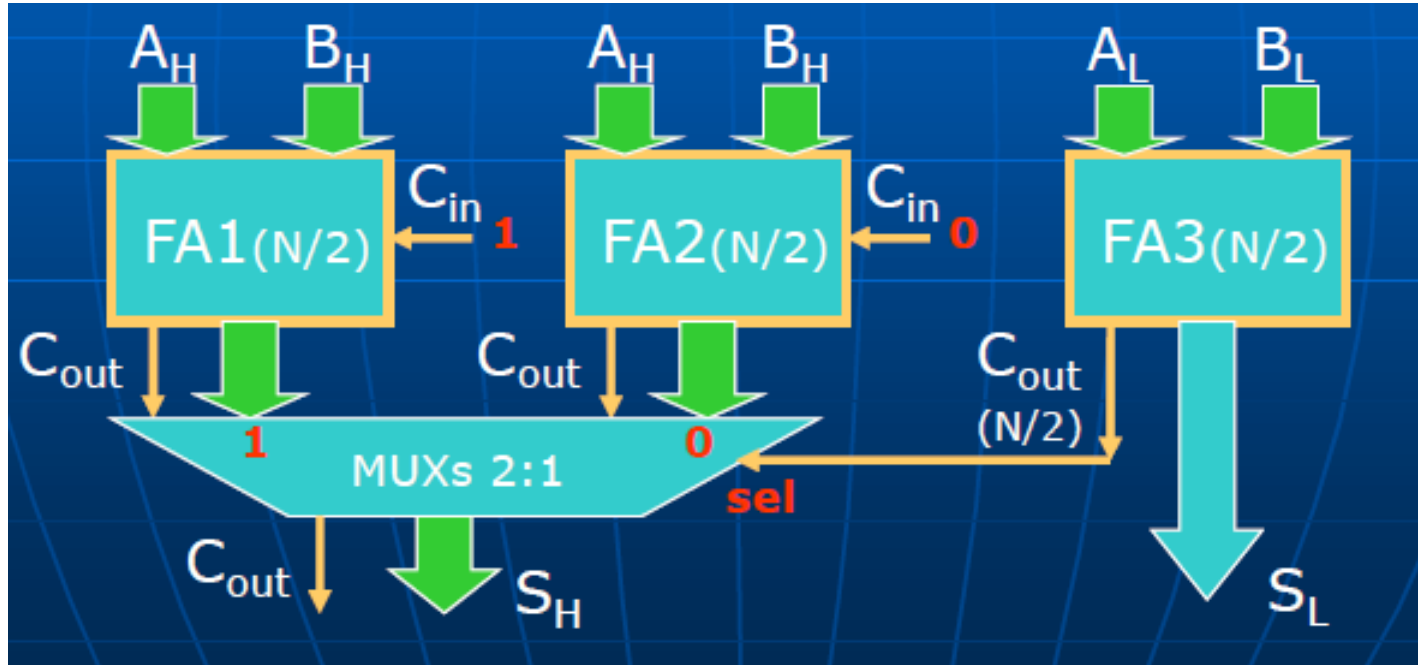
Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	85.93 MHz	85.93 MHz	clock	

N= 64

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax		
1	61.57 MHz	61.57 MHz		

Total logic elements	273 / 29,440 (< 1 %)
Total combinational functions	263 / 29,440 (< 1 %)
Dedicated logic registers	192 / 29,440 (< 1 %)
Total registers	192

Circuitos sumadores: Carry Select



- Esta arquitectura de sumador se basa en dividir los operandos en dos. La suma de los bits LOW se hace directamente, y la suma de los bits HIGH se hace dos veces, una considerando Carry= 1 y otra considerando Carry=0.
- Por último se selecciona la correcta considerando el Carry de la suma LOW

Circuitos sumadores: Suma directa con librerías

- Habiendo dicho todo esto Quartus nos provee de una forma directa de sumar operandos, que esta además optimizada para la arquitectura particular.
- Para usarla no se pueden hacer operaciones directamente sobre los `std_logic_vector`, sino que se utiliza la librería `numeric_std`.
- Los operandos se convierten primero a unsigned (o signed), se hace la operación y finalmente pueden convertirse nuevamente a `std_logic_vector`.

Circuitos sumadores: Suma directa con librerías

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity SumadorGenerico is
    generic ( N : integer := 4 );

    port
    (
        A : in std_logic_vector (N-1 downto 0);
        B : in std_logic_vector (N-1 downto 0);

        S : out std_logic_vector (N-1 downto 0)
    );
end entity;

architecture sumador of SumadorGenerico is
begin

    S <= std_logic_vector ( unsigned(A) + unsigned(B) );

end sumador;

```

Librería numeric std

Ejemplo N=64

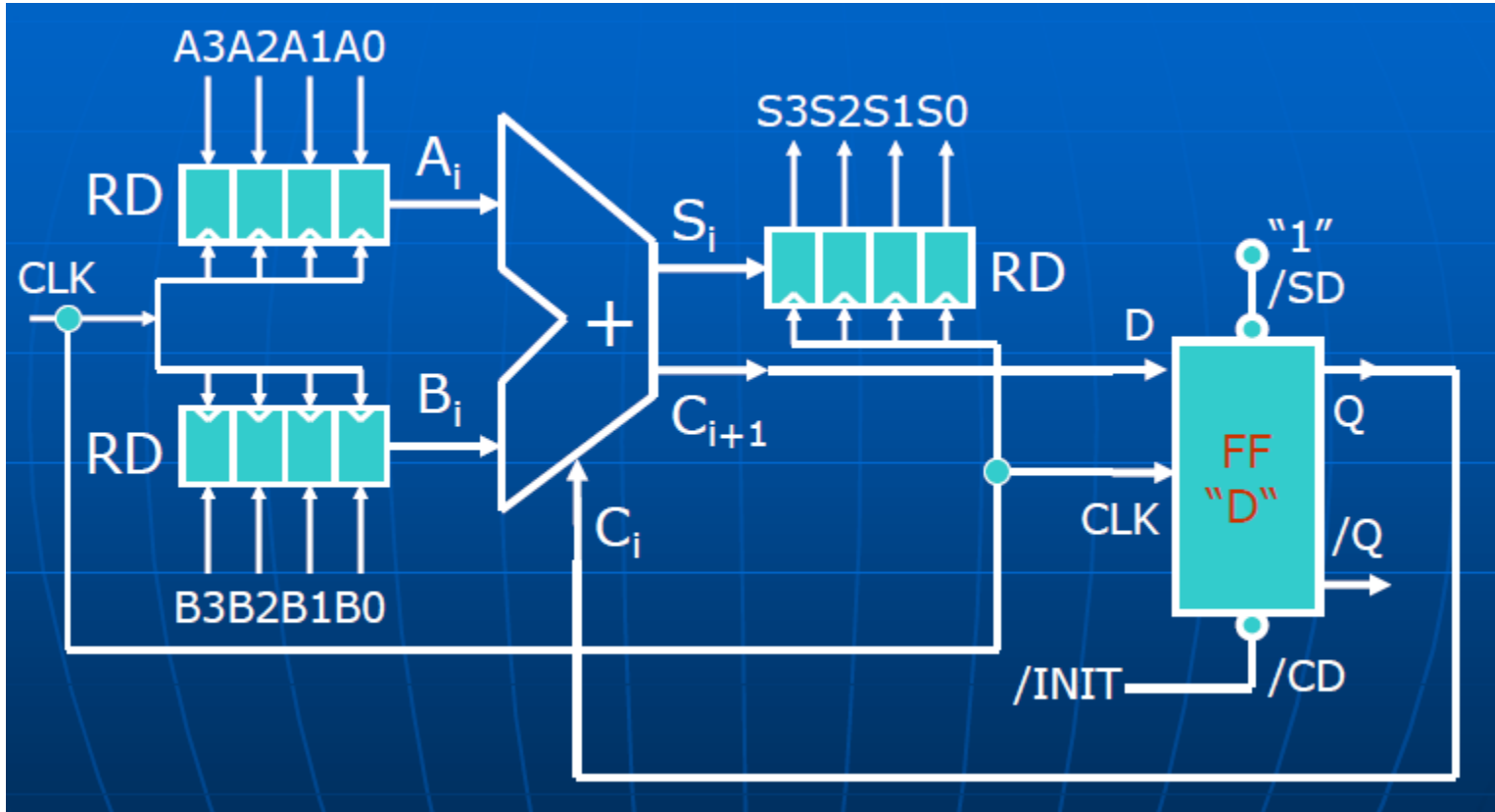
Slow 1200mV 85C Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	179.57 MHz	179.57 MHz	clock	

Total logic elements	192 / 29,440 (< 1 %)
Total combinational functions	64 / 29,440 (< 1 %)
Dedicated logic registers	192 / 29,440 (< 1 %)
Total registers	192

Suma directa

Circuitos sumadores: sumador secuencial



Una arquitectura como esta permite operar el reloj a mayor velocidad, a costa de una mayor latencia (más ciclos de reloj para obtener el resultado)

Circuitos sumadores: sumador secuencial

```

signal salida_A, salida_B : std_logic;
signal C_actual : std_logic := '0';
signal C_sig : std_logic := '0';

signal salida_S : std_logic;

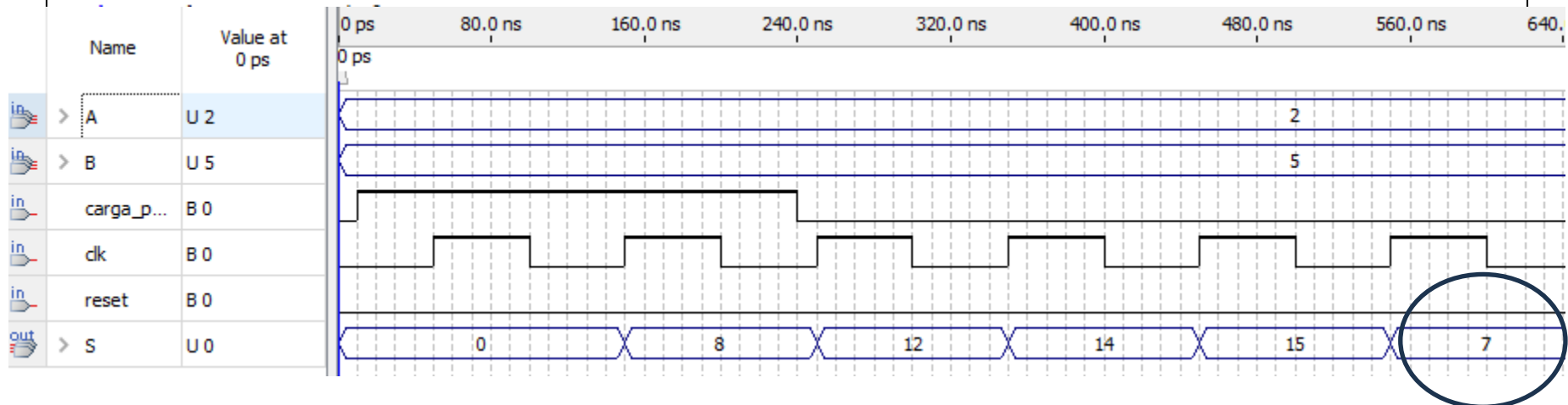
begin

RA: RegistroDesplazamiento generic map (N) port map (clk,reset,'0','1',carga_paralela,A,salida_A);
RB: RegistroDesplazamiento generic map (N) port map (clk,reset,'0','1',carga_paralela,B,salida_B);
RC: RegistroSerieParalelo generic map (N) port map (clk,reset,'0','1',salida_S,S);

FAd: FA port map (salida_A,salida_B,C_actual,salida_S,C_sig);

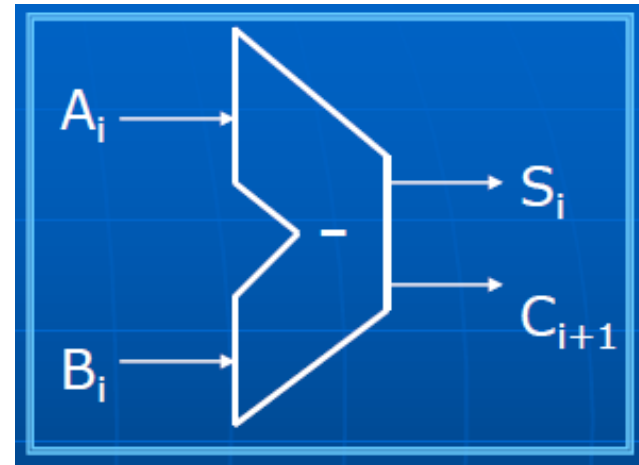
FFd: FF_D port map (C_sig,clk,'0',reset,'1',C_actual,open);

```



Circuitos restadores: Full subtractor

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



El Carry en el restador en realidad sería un “Borrow”, pero se mantiene el nombre por simplicidad

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = \overline{A_i} B_i + \overline{A_i} C_i + B_i C_i$$

¡Mismas expresiones que el sumador con A negado!

Circuitos restadores: Ripple carry

- Los full subtractor se pueden unir en arquitectura Ripple Carry en forma equivalente al sumador:

```
entity FSub is
port
(
  A : in std_logic;
  B : in std_logic;

  Ci : in std_logic;

  S : out std_logic;
  Co : out std_logic
);
```

```
end entity;
```

```
architecture restar of FSub is
begin
```

```
  S <= A xor B xor Ci;
  Co <= (not(A) and B) or (not(A) and Ci) or (B and Ci);
```

```
end restar;
```

```
begin
```

```
  C(0) <= '0';
```

```
  gen:
```

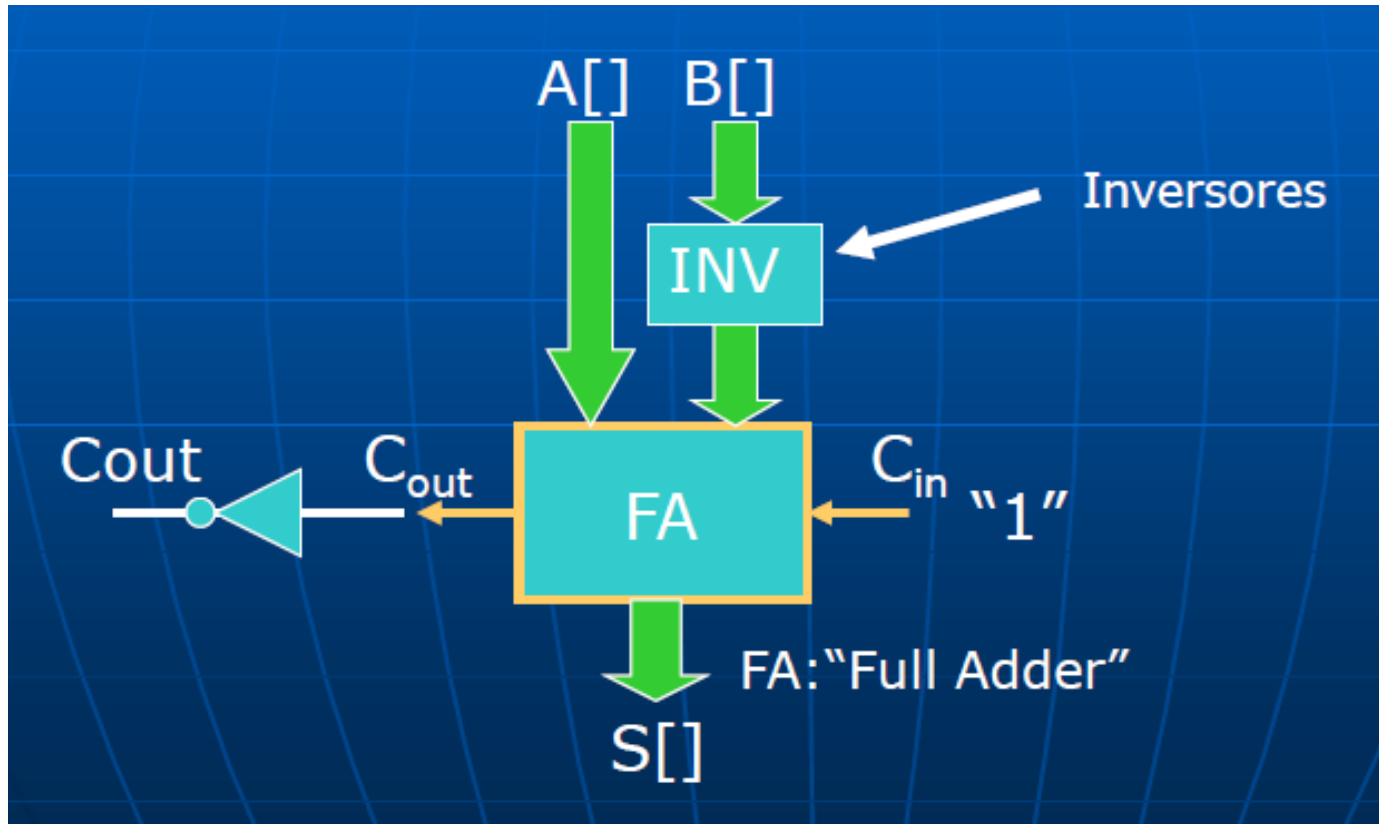
```
  for i in 0 to N-1 generate
```

```
    fa_i: Fsub port map ( A(i),B(i),C(i),S(i),C(i+1));
```

```
  end generate;
```

```
end arch;
```

Circuitos restadores: basado en CA2



Circuitos multiplicadores: números constantes

- Para implementar multiplicaciones en hardware deben distinguirse dos casos que se solucionan de formas muy diferentes.
- Por un lado está el caso de que alguno de los operandos sea constante.
- Si este número es además una potencia de 2 la cosa se vuelve un simple desplazamiento de a izquierda de bits.
- Otros casos pueden resolverse con una combinación de sumas restas y desplazamientos. Por ejemplo:

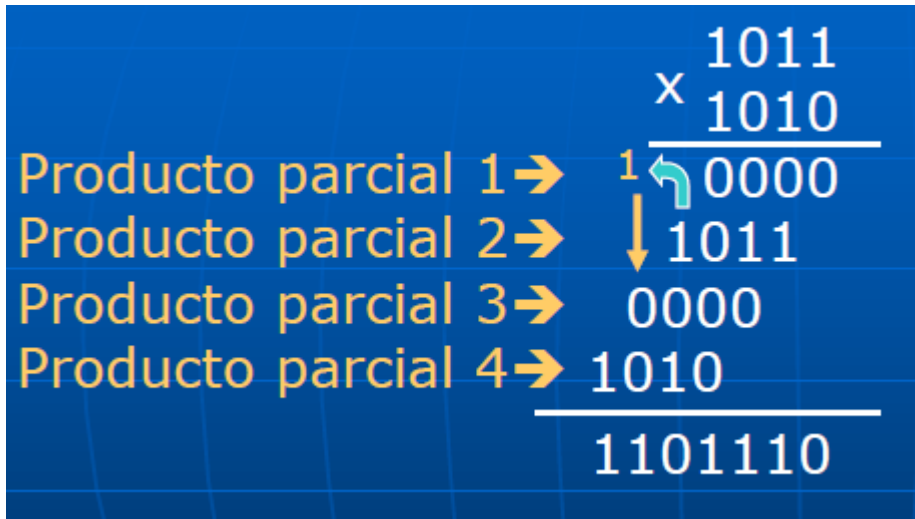
$$3A = 2A + A \quad \text{Desplazamiento de 1 bit y suma}$$

$$5A = 4A + A \quad \text{Desplazamiento de 2 bits y suma}$$

$$15A = 16A - A \quad \text{Desplazamiento de 4 bits y resta}$$

- Con estrategias similares se pueden encontrar formas de computar de forma sencilla cualquier multiplicación por números constantes

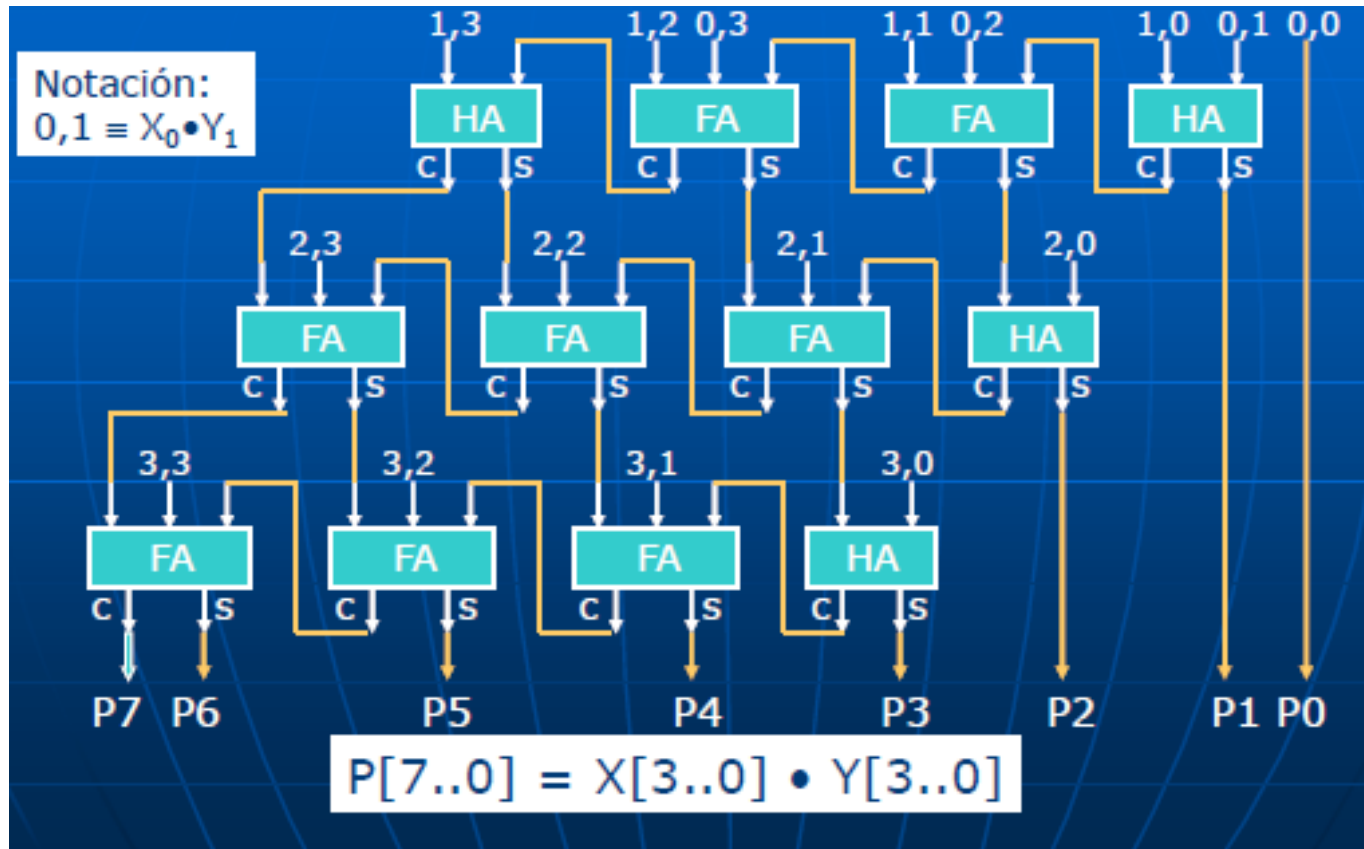
Circuitos multiplicadores: algoritmo tradicional



Hay que hacer el AND entre los bits del multiplicando y los del multiplicador, y luego realizar las sumas teniendo en cuenta los pesos de cada bit mediante operaciones de desplazamiento a la izquierda.

Circuitos multiplicadores: algoritmo tradicional

Para 4 bits toda esta operación resulta algo engorrosa:



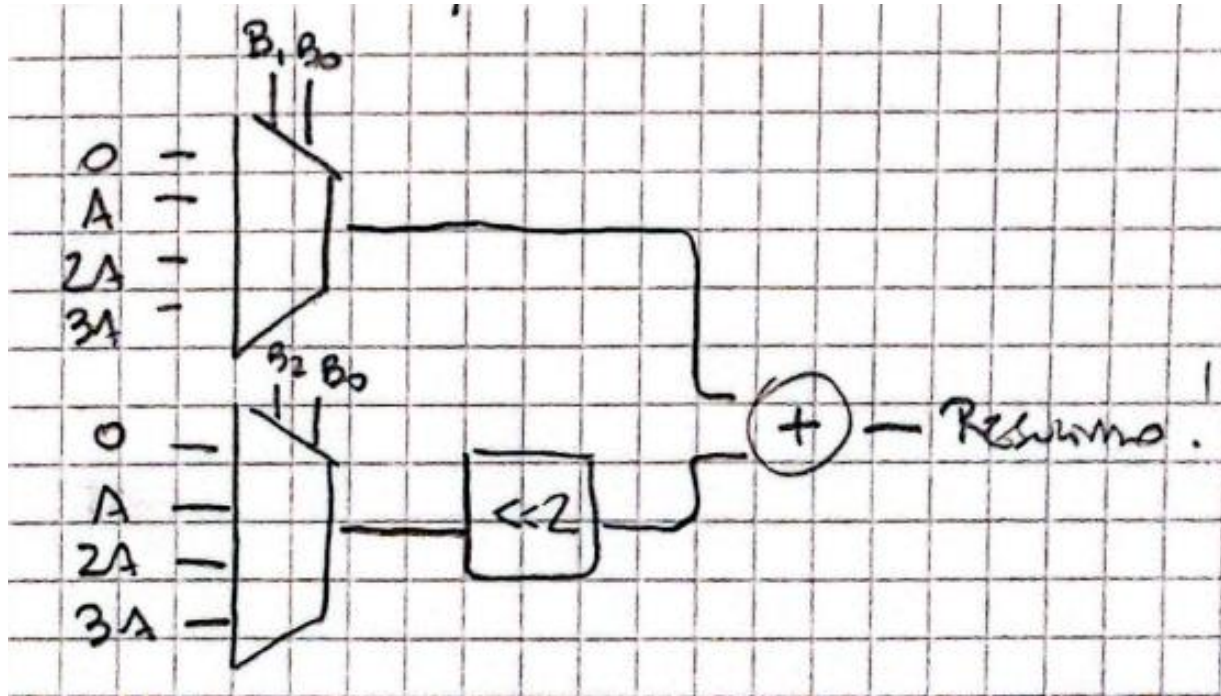
Circuitos multiplicadores: algoritmo de Booth

$$\begin{array}{r}
 \times \quad A_3 \ A_2 \ A_1 \ A_0 \\
 \quad B_3 \ B_2 \ B_1 \ B_0
 \end{array}$$

$A_3 \ A_2 \ A_1 \ A_0$	
$B_3 \ B_2 \ 0 \ 0$	
4 posibilidades	
$B_3 \ B_2$	Resultado
00	0
01	4A
10	8A
11	12A
$\equiv 4 \begin{pmatrix} A \\ 2A \\ 3A \end{pmatrix}$	

$A_3 \ A_2 \ A_1 \ A_0$	
$\times \quad B_1 \ B_0$	
Solo 4 posibilidades!	
$B_1 \ B_0$	Resultado:
00	0000
01	$A_3 \ A_2 \ A_1 \ A_0 = A$
10	$A_3 \ A_2 \ A_1 \ A_0 0 = 2A$
11	$\quad \quad \quad = 3A$

Circuitos multiplicadores: algoritmo de Booth



Circuitos multiplicadores: algoritmo de Booth

```
entity Clase5_c is
port (
    A : in std_logic_vector (3 downto 0);
    B : in std_logic_vector (3 downto 0);
    M : out std_logic_vector (7 downto 0)
);
end entity;

architecture arch of Clase5_c is
    signal zeroA,unA,dobleA,tripleA : std_logic_vector (5 downto 0);
    signal mult1,mult2 : std_logic_vector (5 downto 0);
    signal mult1_ext,mult2_ext : std_logic_vector (7 downto 0);
```

```
component RippleCarry is
    generic ( N: integer := 4);
    port ( A : in std_logic_vector(N-1 downto 0);
          B : in std_logic_vector(N-1 downto 0);
          S : out std_logic_vector(N-1 downto 0));
end component;

begin

zeroA <= "000000";
unA <= "00" & A;
dobleA <= '0' & A & '0';

sum3: RippleCarry generic map(6) port map (unA,dobleA,tripleA);

mult1 <=
    zeroA when ( B(1) = '0' and B(0) = '0' ) else
    unA when ( B(1) = '0' and B(0) = '1' ) else
    dobleA when ( B(1) = '1' and B(0) = '0' ) else
    tripleA when ( B(1) = '1' and B(0) = '1' ) else
    (others => '0');

mult2 <=
    zeroA when ( B(3) = '0' and B(2) = '0' ) else
    unA when ( B(3) = '0' and B(2) = '1' ) else
    dobleA when ( B(3) = '1' and B(2) = '0' ) else
    tripleA when ( B(3) = '1' and B(2) = '1' ) else
    (others => '0');

mult1_ext <= "00" & mult1;
mult2_ext <= mult2 & "00";
sum: RippleCarry generic map(8) port map (mult1_ext,mult2_ext,M);

end arch;
```